

What Paging Actually Costs: A Kernel-Level Study of Paged KV Caches for LLM Decoding

Emaan Heidari, Ara Esfarjani, Kamsi Nwabueze
Ming Hsieh *Department of Electrical and Computer Engineering*
University of Southern California, Los Angeles, CA, USA
ehaidari@usc.edu, aesfarja@usc.edu, knwabuez@usc.edu
github.com/emaanh/paged-attention

Abstract—Paged KV caches remove the internal fragmentation that limits how many sequences an LLM serving system can hold in memory, but the kernel-level price of the block table indirection they introduce is normally visible only as part of end-to-end system throughput. To measure that price directly, we implement a split-K flash-decode attention kernel together with two KV cache managers, one contiguous and one paged, behind a single templated accessor, so that memory layout is the only variable between them. On an NVIDIA A40 we find that paging holds 4 to 8 times more concurrent sequences in a fixed 4 GB budget at a kernel throughput cost of 1.79 times. That cost does not change anywhere between 16 and 256 tokens per page, which localizes it to per-element address resolution rather than to page boundary crossings, and makes page size a pure capacity knob: implementers should choose the smallest page that keeps per-sequence waste acceptable. The flash-decode kernel itself reaches 476 GiB/s, or 73% of attainable bandwidth, against 17 GiB/s for a tuned three-kernel baseline whose single-block softmax is a serial $\mathcal{O}(T)$ stage. We report 1.79 times as an upper bound and enumerate the measurement confounds that bound it.

Index Terms—GPU, CUDA, attention, LLM inference, KV cache, memory management, roofline

I. INTRODUCTION

Large language model inference has two phases with sharply different performance characteristics. During *prefill*, the full prompt is processed in parallel. During *decoding*, tokens are generated one at a time, and each step attends over the entire KV cache accumulated so far. Decoding is therefore bound by memory bandwidth rather than by arithmetic. As contexts lengthen and serving systems handle more concurrent requests, two bottlenecks dominate: kernel efficiency and memory fragmentation. This paper isolates and quantifies both.

A. The Kernel Bottleneck

FlashAttention and FlashAttention-2 [2], [3] are designed for prefill. They tile across the query matrix $\mathbf{Q} \in \mathbb{R}^{T \times d}$ to minimize traffic to high bandwidth memory. At decode time the query collapses to a single vector $\mathbf{q} \in \mathbb{R}^d$, so rather than a full $\mathbf{QK}^\top$ product we need only

$$\mathbf{o} = \text{softmax}\left(\frac{\mathbf{qK}^\top}{\sqrt{d}}\right) \mathbf{V}, \quad (1)$$

where $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d}$ grow by one row per generated token. The only parallelism available is over the token dimension

T . A naive multi-kernel pipeline serializes at the softmax boundary no matter how large T becomes, leaving most of the GPU idle. Flash decoding [4] resolves this by splitting the T tokens across independent GPU blocks and merging their partial softmax results, converting a serial bottleneck into a bandwidth-bound parallel scan.

B. The Fragmentation Bottleneck

Conventional KV cache implementations reserve a contiguous region of GPU memory per sequence, sized to `max_seq_len`. Memory left unused by shorter sequences cannot be reclaimed. At 75% fragmentation, a realistic operating point whenever `max_seq_len` is set conservatively, three of every four reserved bytes are wasted, which directly caps serving concurrency. PagedAttention [5] removes this waste by allocating in fixed-size pages so that each sequence consumes only what it needs. The cost is an extra level of address indirection through a per-sequence block table.

C. Motivation

These bottlenecks are not merely theoretical. PagedAttention has never been implemented in llama.cpp [7], despite a feature request open since 2023. Issue #10860 [8] documents the symptom directly: throughput falls from 110 to 50 tokens per second as the number of parallel slots increases, a consequence of a unified KV cache forcing all sequences to share one contiguous region. A recent pull request [9] introduced per-sequence buffers, which removes cross-sequence attention waste but leaves internal fragmentation unsolved. Before modifying a production codebase, an implementer needs precise kernel-level data on what paging actually costs. This paper supplies that data.

D. Contributions

- 1) **Three attention implementations** evaluated head to head across sequence lengths up to $T = 2,097,152$: a CPU reference, a parallel GPU baseline, and a split-K flash-decode kernel. We show the baseline’s throughput *decreases* with T and explain why (Section VII-B).
- 2) **Two KV cache managers**, `ContiguousPool` and `PagedPool`, behind one kernel interface so that memory layout is the only variable between them.

TABLE I
SUMMARY OF RESULTS. ALL THROUGHPUT FIGURES ARE EFFECTIVE
MEMORY THROUGHPUT IN GiB/s AT $d = 128$ ON ONE A40.

Measurement	Baseline	Flash	Ratio
Kernel, $T=4096$	32.7	146.4	$4.5\times$
Kernel, $T=2,097,152$	17.0	476.2	$28\times$
End to end, $T=2,097,152$	6.4	10.1	$1.6\times$
Fraction of 648 GiB/s peak	2.6%	73%	
Measurement	Contiguous	Paged	Ratio
Kernel, $N=64$, $T=512$	21.5	12.0	$0.56\times$
Sequences in 4 GB, 75% frag.	2048	8192	$4\times$
Sequences in 4 GB, 87% frag.	1024	8192	$8\times$
Waste per sequence	up to 3584 tok	<16 tok	

3) **Systematic sweeps** over split-K block count, fragmentation level, and page size, with throughput and capacity measured independently. The page size sweep yields our least expected result: throughput is completely flat across page sizes, which localizes the paging overhead to per token address resolution rather than to page boundary crossings (Section VII-G).

E. Hypotheses

H1. Paged allocation supports 1.5 to 3 times more concurrent sequences within a fixed memory budget by eliminating fragmentation.

H2. Block table indirection adds 15 to 25% per-request latency overhead, and by Amdahl’s law this cost becomes negligible at high concurrency.

Both are evaluated quantitatively in Section VI and revisited in Section VII. Table I previews the outcome.

II. RELATED WORK

a) *Transformers and decode attention*: Vaswani et al. [1] introduced scaled dot product attention. At decode time the query collapses to a single vector, which makes the operation bandwidth bound: 0.5 FLOPs per byte against the A40’s ridge point of roughly 54 FLOPs per byte. We take this roofline analysis as a starting point and measure how close two concrete GPU implementations actually come to the hardware limit.

b) *FlashAttention and FlashAttention-2*: Dao et al. [2], [3] tile the full query matrix in shared memory to cut memory traffic during prefill and training. That strategy does not apply to decoding, where $\mathbf{q}$ is a single vector and the available parallelism lies along T rather than along the query dimension. This distinction is why a separate decode kernel is needed instead of reusing FlashAttention-2 directly.

c) *Flash decoding*: Dao et al. [4] introduced split-K for single-query decode, dividing T tokens across independent blocks and merging partial softmax results. Our kernel implements this algorithm. Where the original work targets long-context inference inside a complete serving system, we isolate the kernel in a controlled single-GPU microbenchmark and sweep the split-K block count empirically to locate the optimum for the A40.

d) *PagedAttention and vLLM*: Kwon et al. [5] introduced paged KV caches in vLLM, targeting multi-GPU production serving with continuous batching and preemption. Their evaluation reports system-level throughput and latency on real request traces. Our work is complementary: we isolate the single-GPU kernel-level cost of paged indirection and measure directly how page size, fragmentation level, and block table lookup interact. This is the fine-grained data that system-level evaluations necessarily aggregate away.

e) *Online softmax*: Milakov and Gimelshein [6] showed that softmax can be computed in a single pass using running maximum and sum statistics rather than materializing all scores. Flash decoding depends on this result to make per-chunk partial outputs composable without a global barrier, and our implementation follows their formulation directly.

III. TECHNICAL APPROACH

A. Attention Kernels

a) *CPU reference*: A scalar C++ implementation of Eq. 1, used only for correctness checking. Every GPU result in this paper is validated against it.

b) *Parallel GPU baseline*: Three CUDA kernels connected by implicit launch barriers. First, one thread per token computes $s_t = \mathbf{q} \cdot \mathbf{k}_t / \sqrt{d}$ with a strided loop over head dimensions. Second, a single 128-thread block computes a numerically stable softmax by parallel tree reduction over all T scores. Third, one thread per output dimension accumulates $\sum_t w_t \mathbf{v}_{t,i}$. Each kernel is individually well optimized, using tree reduction, coalesced access, and shared memory for intermediates. The limitation is structural rather than a matter of tuning: softmax must complete over all T scores before output accumulation can begin, so the pipeline is serial across that boundary no matter how large T grows.

c) *Flash-decode kernel*: Algorithm 1 gives the structure and Fig. 1 shows the dataflow. The token sequence is split across B blocks that each process one chunk independently. Every block runs an online softmax [6], so it never needs to observe scores belonging to another block and no global barrier is required during the parallel phase.

B. KV Memory Layout

Both pools are consumed through the same templated accessor, so identical kernel code serves contiguous and paged memory and the layout is the only variable between the two configurations.

```

struct ContiguousKV {
    const float *K, *V;
    __device__ float key(int token, int dim, int d)
    { return K[token * d + dim]; }
};

struct PagedKV {
    const float *K_pool, *V_pool;
    const int *block_table; int page_size;
    __device__ float key(int token, int dim, int d) {
        int page = block_table[token / page_size];
        return K_pool[(page*page_size + token%page_size)*d
            + dim];
    }
};

```

Algorithm 1 Split-K flash decode

Require: query q , cache K, V , length T , blocks B

Phase 1: partial kernel, B blocks in parallel

```

1: for block  $b = 0$  to  $B - 1$  do
2:    $lo \leftarrow \lfloor bT/B \rfloor, \quad hi \leftarrow \lfloor (b+1)T/B \rfloor$ 
3:    $m \leftarrow -\infty, \quad \ell \leftarrow 0, \quad o \leftarrow 0$ 
4:   for each tile of 128 tokens in  $[lo, hi]$  do
5:      $s_t \leftarrow q \cdot k_t / \sqrt{d}$   $\triangleright$  one thread per token
6:      $m' \leftarrow \max(m, \max_t s_t)$   $\triangleright$  tree reduction
7:     rescale  $\ell, o$  by  $e^{m-m'}$ ; accumulate tile
8:      $m \leftarrow m'$ 
9:   end for
10:  emit  $(o_b, m_b, \ell_b)$  to global memory
11: end for

```

Phase 2: reduce kernel, 1 block of d threads

```

12:  $m^* \leftarrow \max_b m_b$ 
13:  $o \leftarrow \sum_b o_b e^{m_b - m^*}, \quad \ell \leftarrow \sum_b \ell_b e^{m_b - m^*}$ 
14: return  $o/\ell$ 

```

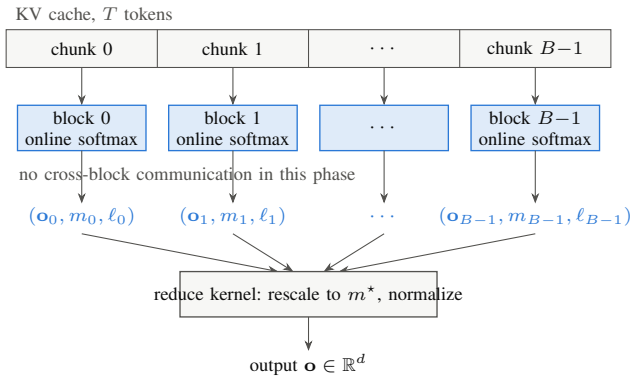


Fig. 1. Split-K flash decode dataflow. Each of the B blocks owns a disjoint chunk of the KV cache and runs an online softmax over it, emitting an unnormalized partial output together with its running maximum and sum. Because those triples are composable, the parallel phase needs no global barrier; a single reduce kernel rescales every partial to a common maximum and normalizes once.

Listing 1. KV accessors (kv_layout.hpp)

ContiguousKV resolves an address with a single multiply and add. PagedKV must first load the physical page index from the block table, so every element access carries a dependent global load on its critical path. Fig. 2 contrasts the two address paths, and Section VII-F shows that this is where the measured throughput penalty originates.

C. KV Cache Pools

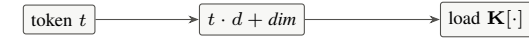
Both managers implement one interface, so the kernel is unaware of which is in use:

```

class KVPool {
public:
  virtual int admit(const float* K, const float* V, int len) = 0;
  virtual void release(int slot) = 0;
  virtual void append_token(int slot, int len,

```

ContiguousKV



one multiply and add, no dependent load

PagedKV

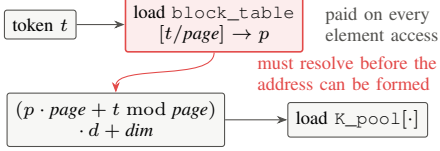


Fig. 2. Address resolution in the two accessors. The paged path inserts a block table load that must complete before the KV address can even be formed. For a bandwidth-bound kernel this dependent load sits directly on the critical path, and because it is incurred on every element access rather than once per page, its cost is independent of page size.

```

const float* k, const float* v
) = 0;
virtual void decode(int slot, const float* q, float* out
) = 0;
};

```

a) *ContiguousPool*: Reserves $\text{max_seq_len} \times d$ contiguous floats per slot at admission time. Simple and fast, but it wastes $\text{max_seq_len} - \text{actual_len}$ tokens of capacity in every slot.

b) *PagedPool*: Divides the pool into fixed-size pages of page_size tokens, tracked by a free list. A sequence claims pages as it grows and returns them on release, and a per-slot *block table* records which physical page backs each logical token range. Worst-case waste is $\text{page_size} - 1$ tokens per sequence regardless of max_seq_len . Fig. 3 contrasts the two layouts.

IV. PARALLELIZATION DESIGN

a) *Thread decomposition*: The partial kernel launches B blocks of 128 threads. Block b owns tokens $[bT/B, (b+1)T/B]$ with no overlap, so during the parallel phase blocks carry no data dependencies on one another.

b) *Breaking the softmax dependency*: Standard attention requires all T scores before it can normalize. Online softmax removes that requirement: each block maintains a running maximum and running sum and updates them tile by tile, so no cross-block communication is needed until the partial kernel has finished. The reduce kernel then merges the B partial results in one block of d threads.

c) *Synchronization*: Inside a block, `__syncthreads()` is called after the query is staged into shared memory and after each of the two parallel reductions. Between the partial and reduce kernels, CUDA's sequential launch guarantee supplies the barrier, so no explicit fence is needed.

d) *Load balance*: Chunk sizes differ by at most one token, $\lfloor T/B \rfloor$ against $\lceil T/B \rceil$, which keeps blocks within a single memory transaction of one another. The reduce kernel touches $B \times d$ values in total, negligible beside the partial kernel's $2Td$ bytes.

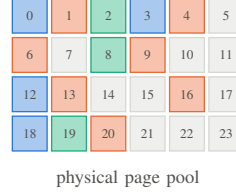
(a) ContiguousPool

every slot reserves max_seq_len , so the unused tail is unreclaimable



(b) PagedPool

pages are claimed on demand, so waste is bounded by one partial page per sequence



block_table for seq 0:

0	3	12	18
---	---	----	----

logical token t lives in physical page $\text{block_table}[t/\text{page_size}]$, so the four shaded pages above are seq 0's

■ seq 0 ■ seq 1 ■ seq 2
grey pages are free

Fig. 3. Contiguous versus paged KV allocation. Under (a), admission reserves a full max_seq_len slot per sequence and the hatched remainder is unrecoverable, so capacity is governed by the configured maximum rather than by actual demand. Under (b), each sequence holds only the pages it has filled and a block table maps logical token ranges onto arbitrary physical pages; the only waste is the partial final page. The cost of (b) is the extra indirection shown in Fig. 2.

TABLE II
PLATFORM AND KERNEL CONFIGURATION.

GPU	NVIDIA A40, 48 GB GDDR6
Peak bandwidth	696 GB/s (648 GiB/s)
Peak FP32 throughput	37.4 TFLOP/s
Ridge point	53.7 FLOP/byte
SM count	84
Host	1 CPU core, 8 GB RAM
Cluster	USC CARC Discovery
Toolchain	NVCC 12, C++17, CMake 3.29
Timing	Google Benchmark v1.8
Correctness	Googletest, tolerance 10^{-4}
FLASH_TILE	128 threads per block
FLASH_BLOCKS	128 split-K blocks (Sec. VI-A)
PAGE_SIZE	16 tokens per page, configurable
Precision	FP32 throughput

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

Table II lists the platform and the fixed kernel parameters. Each block allocates $(d + 2 \times 128) \times 4 = 1536$ bytes of shared memory, comprising 128 floats for the cached query and two 128-float reduction buffers, which is far inside the A40's 48 KB per streaming multiprocessor.

a) *Correctness*: All GPU kernels are validated against the CPU reference for $d \in \{8, 64, 128\}$ and $T \in \{4, 32, 256, 1024\}$, with maximum absolute error below 10^{-4} . Pool tests cover admission, release, slot recycling, overflow, token append across page boundaries, and multi-slot independence. All tests pass.

b) *Benchmarks*:

- **Kernel comparison.** Kernel-only and end-to-end throughput for the CPU reference, the parallel baseline, and flash decode across $T \in [4096, 2097152]$ at $d = 128$.
- **Block count sweep.** Flash-decode latency at $B \in \{16, 32, 64, 128, 256, 512, 1024\}$ over the same range of T .
- **Capacity report.** Sequences fitting a 4 GB budget as max_seq_len varies, with actual_len fixed at 512.
- **Page size sweep.** Paged pool throughput at $N = 64$, $\text{actual_len} = 512$, for page sizes $\{16, 32, 64, 128, 256, 512\}$.

c) *Metrics*: *Kernel throughput* counts only GPU execution time. *End-to-end throughput* additionally includes host to device and device to host transfers. Both are reported as effective memory throughput, computed as bytes of KV touched divided by wall time. Following Google Benchmark's convention these rates are binary, so we write them as GiB/s throughout; the A40's 696 GB/s of peak bandwidth is equivalently 648 GiB/s, and we compare against the latter. *Latency* is raw wall time per benchmark iteration in microseconds.

VI. RESULTS

A. Block Count Sweep

Fig. 4 shows flash-decode latency across all seven block counts and Fig. 5 the resulting peak throughput at $T = 2,097,152$. At $T = 4096$ every configuration behaves identically; divergence first appears at $T = 8192$ among the most under-parallelized configurations. By the largest length, $B = 16$ is 4.5 times slower than $B = 128$. The optimal window is $B \in [128, 256]$, which tie within 0.5% (476.6 against 474.4 GiB/s). Above 256 throughput falls away as chunks become too small for efficient streaming and the reduce kernel has more partials to merge.

Result: $B = 128$ is optimal on the A40, and we adopt it as the default for all subsequent measurements.

B. Kernel Throughput

Fig. 6 shows kernel-only throughput for flash decode and the parallel baseline. Two opposite trends stand out. Flash-decode throughput climbs steadily from 146 to 476 GiB/s as T grows, meaning the kernel becomes better at saturating memory bandwidth as sequences lengthen. The baseline does the reverse, falling from 33 to 17 GiB/s. The cause is not implementation quality, as it uses tree reduction and coalesced access throughout, but algorithmic structure: its softmax kernel runs on one 128-thread block regardless of T , creating an $\mathcal{O}(T)$ serial stage that grows linearly with sequence length.

C. End-to-End Throughput

Fig. 7 reports end-to-end throughput, which includes host to device transfer of the full KV cache and device to host transfer of the output. This models the case where KV data

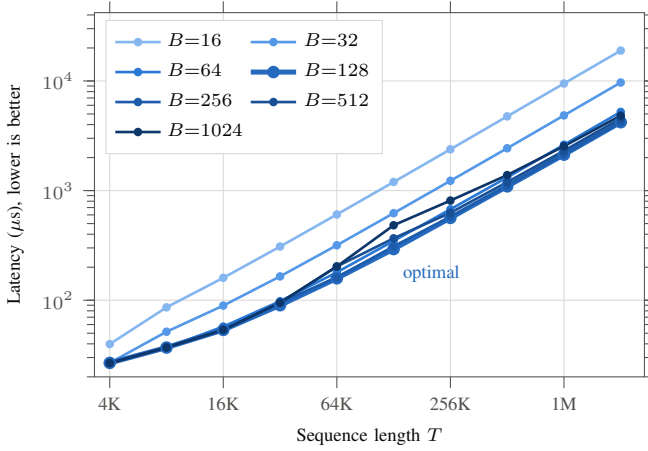


Fig. 4. Flash-decode latency against sequence length for all seven split-K block counts at $d = 128$. Colour encodes block count from light (16) to dark (1024); the emphasized curve is the chosen default. Too few blocks under-parallelizes and too many over-shards the sequence into chunks that stream inefficiently.

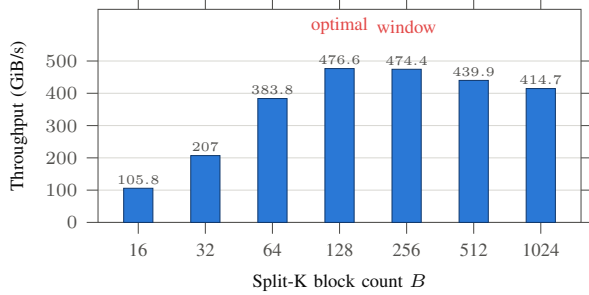


Fig. 5. Peak flash-decode throughput per block count at $T = 2,097,152$, $d = 128$. The 128 to 256 window is near optimal. At $B = 16$ the kernel reaches only 105.8 GiB/s, leaving roughly 84% of attainable bandwidth unused.

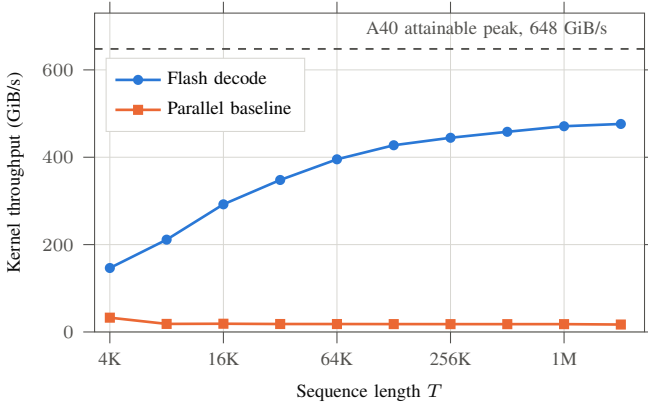


Fig. 6. Kernel-only throughput against sequence length. Flash decode scales toward the bandwidth ceiling, reaching 73% of attainable peak, while the parallel baseline degrades because its single-block softmax becomes an $\mathcal{O}(T)$ serial bottleneck.

lives on the CPU, and it is therefore pessimistic relative to a real serving system in which the cache is resident on the GPU.

The CPU reference exposes the host memory hierarchy in

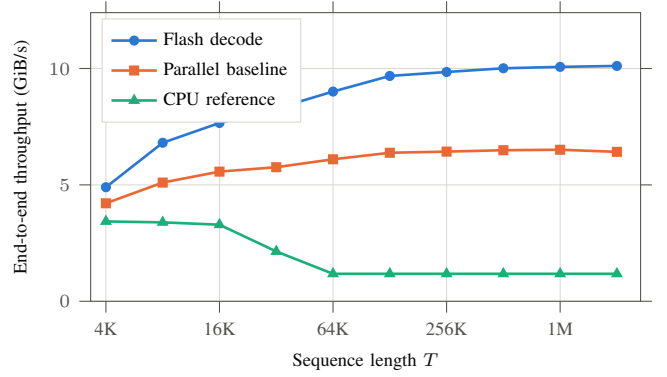


Fig. 7. End-to-end throughput including transfer over PCIe. Flash decode leads the baseline by 1.6 times at the largest length. The wide gap against Fig. 6 is transfer cost, quantified in Section VII-E.

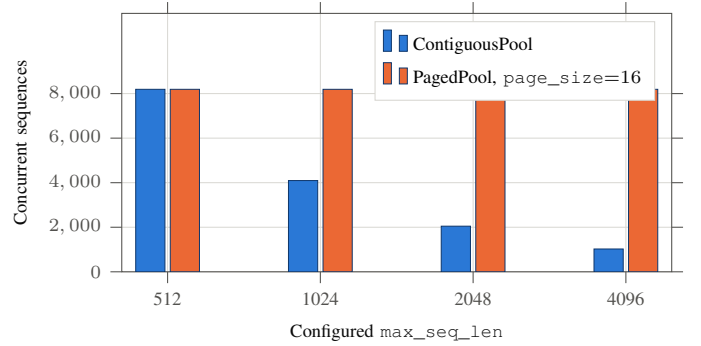


Fig. 8. Sequences fitting a 4 GB KV budget at $\text{actual_len} = 512$, $d = 128$. At $\text{max_seq_len} = 2048$, which is 75% fragmentation, the paged pool holds 4 times more sequences; at 4096 the advantage reaches 8 times.

two stages. Throughput holds near 3.3 GiB/s for $T \leq 16384$ while the KV working set fits in L3, drops to 2.14 GiB/s at $T = 32768$ as that set spills, and settles at 1.18 GiB/s for all larger T once the reference is fully DRAM bound. Flash decode grows from 4.9 to 10.1 GiB/s, far below its 476 GiB/s kernel figure because transfer dominates at every length. The baseline trails flash by 1.6 times at the largest T .

D. Memory Capacity Under Fragmentation

Fig. 8 shows how many sequences fit a 4 GB KV budget at $\text{actual_len} = 512$, $d = 128$. The paged pool holds 8192 sequences irrespective of the configured maximum, while the contiguous pool halves each time max_seq_len doubles. Kernel throughput per token is unaffected by this choice, so the entire cost of fragmentation lands on serving capacity.

E. Page Size Sweep

Fig. 9 shows paged pool throughput across page sizes at $N = 64$, $\text{actual_len} = 512$. Every page size from 16 through 256 delivers the same 12.0 GiB/s. Only $\text{page_size} = 512$, which degenerates to a single page per sequence, recovers the contiguous baseline of 21.5 GiB/s. Waste per sequence meanwhile grows linearly with page size, from 15 tokens to 255.

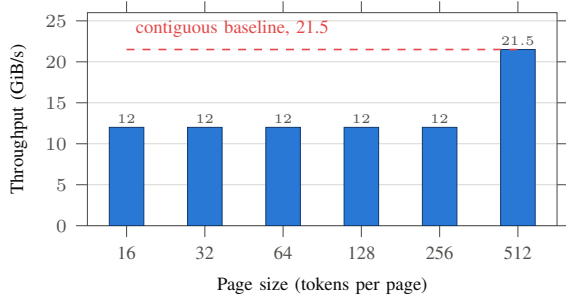


Fig. 9. Page size sweep at $N = 64$, $\text{actual_len} = 512$, $d = 128$. Throughput is flat across every genuinely paged configuration. The 512 bar reaches the contiguous line only because one page then covers the whole sequence, removing the indirection entirely.

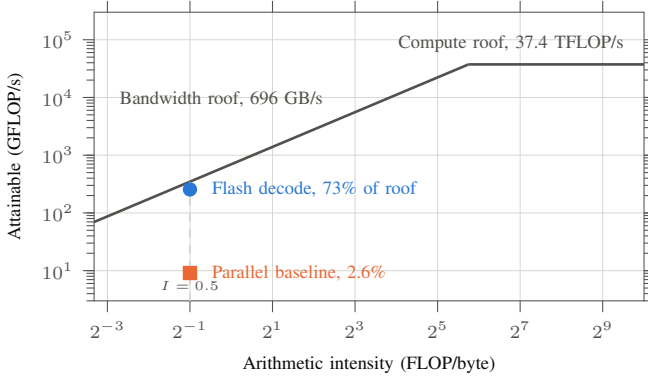


Fig. 10. Roofline for the A40 with both kernels plotted at the decode arithmetic intensity of Eq. 2. The ridge point lies at 53.7 FLOP/byte, two orders of magnitude to the right of where decode attention operates, so the only headroom available is bandwidth headroom. Flash decode captures 73% of it.

VII. ANALYSIS AND DISCUSSION

A. Decode Attention is Bandwidth Bound

For one decode step over T tokens at head dimension d , the kernel reads $\mathbf{K}$ and $\mathbf{V}$, that is $2Td$ floats, and touches a further $2d$ floats for the query and output. Total traffic is therefore roughly $8Td$ bytes in FP32, against roughly $4Td$ floating-point operations for the dot products and the weighted accumulation:

$$I = \frac{4Td}{8Td} = 0.5 \text{ FLOP/byte.} \quad (2)$$

The A40’s ridge point is $37.4/0.696 \approx 53.7$ FLOP/byte. Since $0.5 \ll 53.7$, decode attention sits far to the left of the ridge and is firmly bandwidth bound, as Fig. 10 makes explicit. This is why flash decode approaches but never crosses the bandwidth ceiling: at $T = 2,097,152$ it attains 73% of the achievable roof, and no amount of arithmetic tuning can move it further right.

B. Why the Baseline Degrades with Length

The score and output kernels of the baseline both scale well, using one thread per token and one thread per output dimension respectively, with tree reduction and coalesced access

throughout. The bottleneck is the softmax kernel, which is constrained to a single 128-thread block in order to retain a global view of all T scores. Its runtime is $\mathcal{O}(T)$ on 128 threads, so it doubles whenever T doubles. At $T = 2,097,152$ softmax dominates the pipeline, which is why baseline throughput falls from 33 to 17 GiB/s while flash decode rises. The lesson is not that the baseline is written badly, because it is not, but that a three-kernel pipeline with a global barrier after scoring cannot scale past what a single block can do.

C. Why $B = 128$ is Optimal

Below 64 blocks there are too few blocks to occupy all 84 streaming multiprocessors, and each block must stream a chunk too large to keep in flight efficiently. Above 256 blocks two effects compound: chunks shrink below the size that sustains efficient memory transactions, and the reduce kernel has more partial results to merge. Between those limits, occupancy is saturated while chunks remain large enough to stream well. Figs. 4 and 5 confirm this, with 128 and 256 within 0.5% of one another and 512 and 1024 degrading by 8% and 13% respectively.

D. The CPU Reference Exposes L3

The CPU curve in Fig. 7 traces the host memory hierarchy directly. For $T \leq 16384$ the KV working set, $2Td \times 4$ bytes or about 16 MB, fits in L3 and throughput holds near 3.3 GiB/s. At $T = 32768$ the set reaches roughly 33 MB, spills from L3, and throughput falls to 2.14 GiB/s. By $T = 65536$ the reference is fully DRAM bound and plateaus at exactly 1.18 GiB/s for every larger length, which is the effective main memory bandwidth for this streaming pattern.

E. End to End Against Kernel Only: the Transfer Wall

Flash decode reaches 476 GiB/s of kernel throughput but only 10.1 GiB/s end to end. The difference is PCIe transfer. At $T = 2,097,152$ the KV matrices are $2 \times 2097152 \times 128 \times 4$ bytes, or 2.1 GB. Moving that over PCIe 4.0 at a realistic achieved 12 to 16 GB/s costs 134 to 179 ms of the 198 ms measured end to end, so transfer accounts for most of wall time and the kernel itself for a small minority of it.

This matters only when KV data originates on the host. In a serving system the KV cache is resident on the GPU across decode steps, so the kernel-only figure in Fig. 6 is the correct performance indicator and the end to end figure is a lower bound driven by an interconnect that a real deployment never crosses per step.

F. Why Paged Throughput is 1.79 Times Lower

The paged kernel is slower because `PagedKV::key()` must load `block_table[token / page_size]` from global memory before it can even compute the address of the KV element it wants, as drawn in Fig. 2. This is a load-use dependency: the memory pipeline stalls waiting on the page index before it can issue the load that actually matters. For a kernel whose performance is determined entirely by how fast it can stream bytes, any stall placed on the critical path translates

directly into lost throughput. We measure contiguous at 21.5 GiB/s and paged at 12.0 GiB/s, a ratio of 1.79.

G. Page Size Does Not Affect Throughput

If page boundary crossings were responsible, larger pages would cross them less often and throughput would improve with page size. Fig. 9 shows that it does not: throughput is identical from 16 through 256 tokens per page. The overhead therefore cannot be a function of how frequently a new page is entered. It is instead incurred on every individual element access, because the block table load in Listing 1 is evaluated per access rather than once per page.

Practical consequence. Page size governs memory waste and nothing else. An implementer should pick the smallest page that keeps per-sequence waste tolerable for the expected length distribution, and can do so without paying any throughput penalty for the choice.

H. H1: Serving Capacity, Confirmed and Exceeded

At `max_seq_len = 2048` against `actual_len = 512`, which is 75% fragmentation, the paged pool holds 4 times more concurrent sequences than contiguous in the same 4 GB budget. At 87% fragmentation the advantage grows to 8 times. H1 predicted 1.5 to 3 times, so the measured gain exceeds the predicted upper bound. H1 is confirmed and exceeded.

I. H2: Per-Request Overhead, Magnitude Wrong

H2 predicted 15 to 25% latency overhead per request. The measured ratio of 1.79 corresponds to a 79% overhead, several times the predicted figure, so the magnitude in H2 is wrong. The Amdahl argument embedded in H2, that this cost becomes negligible at high concurrency, remains directionally sound: a fixed 1.79 times kernel cost is amortized against a 4 to 8 times larger population of resident sequences, which is a favourable trade whenever the system is capacity limited rather than latency limited. We did not benchmark concurrent batched decoding, however, so this part of H2 is argued rather than demonstrated.

J. Threats to Validity

Three aspects of the measurement setup bound how far these results generalize, and we state them explicitly rather than leave them implicit.

a) *Per-decode block table synchronization:* The paged decode path copies the active block table from host to device on every call, whereas the contiguous path performs no equivalent transfer. This copy is small, a few hundred bytes, but it is synchronous and it sits inside the timed region. Some fraction of the 1.79 times ratio in Section VII-F is therefore attributable to that transfer rather than to indirection alone, and the reported ratio should be read as an upper bound on the cost of paging.

b) *Sequential physical page assignment:* Pages are drawn from a freshly initialized free list in ascending order, so a sequence admitted into an empty pool receives physically consecutive pages. The paged and contiguous configurations consequently present very similar address streams to the memory system, and the measurements here do not capture the loss of locality that would follow from repeated admission and release cycles fragmenting the pool. Realistic churn would be expected to widen the gap.

c) *Serialized per-sequence decode and FP32:* Decode calls are issued one sequence at a time. Production systems fuse all N sequences into a single kernel launch, which amortizes launch overhead that is included in our per-call measurements and would reduce the paged pool’s relative cost. All kernels are FP32; FP16 would halve KV footprint and double effective bandwidth, moving both pools along the roofline of Fig. 10. Finally, each configuration is reported as a single Google Benchmark estimate without run to run variance, so small differences such as the 0.5% between $B = 128$ and $B = 256$ should not be treated as separable.

VIII. CONCLUSION

Two questions motivated this work: how decode attention kernels are optimized at the GPU level, and what paged memory management actually costs. Both now have concrete answers.

Flash decoding parallelizes over the token dimension instead of serializing at the softmax boundary, converting a pipeline bottleneck into a bandwidth-bound scan that reaches 73% of the A40’s attainable roof and outperforms a well-written three-kernel baseline by 28 times at long context. The baseline’s decline with increasing T is a structural property of its global barrier, not a defect of its implementation.

Paged memory management eliminates internal fragmentation at a cost that is fixed per token access and independent of page size. Concretely:

- **H1 confirmed and exceeded.** Paging supports 4 to 8 times more concurrent sequences in a fixed budget, beyond the predicted 1.5 to 3 times. The gain is entirely in capacity; per-token kernel throughput is unchanged by the fragmentation level.
- **H2 wrong in magnitude.** The paged kernel is 79% slower rather than 15 to 25% slower, a load-use stall incurred on every element access. The claim that this amortizes at concurrency is sound in direction but requires batched evaluation to establish.
- **Unexpected result.** Page size has no measurable effect on throughput. The cost is per token access, not per page boundary, which makes page size a pure capacity knob.

a) *Future work:* The 79% figure deserves to be decomposed rather than accepted. The immediate steps are to hold block tables resident on the device so no per-decode transfer enters the timed region, to hoist the block table load out of the inner loop so it is resolved once per page instead of once per element, to batch the decode across all N sequences in a single launch, and to repeat the page size sweep against a

deliberately fragmented pool so that locality effects are visible. Each of these targets one of the confounds in Section VII-J. The open question that matters most for practitioners is the crossover point at which the capacity advantage of paging outweighs its per-token cost under a realistic distribution of request lengths, which is precisely the data needed to decide whether a PagedAttention implementation in llama.cpp [7] would repay its engineering cost.

ARTIFACT AVAILABILITY

The CUDA kernels, both KV cache managers, the unit test suite, and the complete benchmark harness that produced every figure in this paper are available at github.com/emaanh/paged-attention. Each figure corresponds to a named Google Benchmark target, documented with its invocation in the repository README.

REFERENCES

- [1] A. Vaswani et al., “Attention is all you need,” in *NeurIPS*, vol. 30, 2017.
- [2] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *NeurIPS*, vol. 35, 2022, pp. 16344 to 16359.
- [3] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *ICLR*, 2024.
- [4] T. Dao, D. Haziza, F. Massa, and G. Sizov, “Flash-Decoding for long-context inference,” Stanford CRFM, Oct. 2023. [Online]. Available: <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>
- [5] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *SOSP*, 2023.
- [6] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax,” arXiv:1805.02867, 2018.
- [7] llama.cpp contributors, “Feature request: PagedAttention,” GitHub Issue #1955, Jun. 2023. [Online]. Available: <https://github.com/ggml-org/llama.cpp/issues/1955>
- [8] llama.cpp contributors, “Throughput degradation with parallel slots due to unified KV cache,” GitHub Issue #10860. [Online]. Available: <https://github.com/ggml-org/llama.cpp/issues/10860>
- [9] llama.cpp contributors, “Per-sequence KV buffers to reduce cross-sequence attention waste,” GitHub PR #14363, Jul. 2025. [Online]. Available: <https://github.com/ggml-org/llama.cpp/pull/14363>